Презентация по лабораторной работе №8

Оптимизация

Компьютерный практикум по стат. анализу данных

2026 г.

Российский университет дружбы народов, Москва, Россия

Информация

Докладчик

- Танрибергенов Эльдар
- студент 4 курса из группы НПИбд-01-22
- ФМиЕН, кафедра прикладной информатики и теории вероятностей
- Российский университет дружбы народов

Цели и задачи

Цель работы

Основная цель работа — освоить пакеты Julia для решения задач оптимизации.

Результаты

Линейное программирование

Предположим, что требуется решить следующую задачу линейного программирования:

$$12x + 20y \to \min$$

при заданных ограничениях:

$$6x + 8y \geqslant 100, \quad 7x + 12y \geqslant 120, \quad x \geqslant 0, \quad y \geqslant 0.$$

Рис. 1: Задача

Линейное программирование

```
[3]: # Определение объекта модели с именем model:
     model = Model(GLPK.Optimizer)
```

```
[3]: A JuMP Model
     ├ solver: GLPK
     ├ objective_sense: FEASIBILITY_SENSE
     ├ num_variables: 0
     ├ num_constraints: 0
     └ Names registered in the model: none
```

Рис. 2: Определение объекта модели с именем model

```
[4]: # Определение переменных x, y и граничных условий для них:
     @variable(model, x >= 0)
     @variable(model, y >= 0)
```

[4]: y

Рис. 3: Определение переменных x,y и

```
[5]: # Определение ограничений модели:
     @constraint(model, 6x + 8y >= 100)
     @constraint(model, 7x + 12y >= 120)
```

[5]: $7x + 12y \geq 120$

Рис. 4: Определение ограничений модели

```
[6]: # Определение целевой функции:
     @objective(model, Min, 12x + 20y)
```

[6]: $12x + 20y$

Рис. 5: Определение целевой функции

Результат

```
[7]: # Вызов функции оптимизации:
     optimize!(model)
```

Рис. 6: Вызов функции оптимизации

```
[8]: # Определение причины завершения работы оптимизатора:
     termination_status(model)

[8]: OPTIMAL::TerminationStatusCode = 1
```

Рис. 7: Определение причины завершения работы оптимизатора

```
[10]: # Демонстрация первичных результирующих значений переменных x и y:
      @show value(x);
      @show value(y);
      # Демонстрация результата оптимизации:
      @show objective_value(model);

      value(x) = 14.999999999999993
      value(y) = 1.2500000000000047
      objective_value(model) = 205.0
```

Рис. 8: Результат

Векторизованные ограничения и целевая функция оптимизации

Предположим, что требуется решить следующую задачу:

$$\vec{c}^T\vec{x} \to \min$$

при заданных ограничениях:

$$A\vec{x} = \vec{b}, \quad \vec{x} \succeq 0, \quad \vec{x} \in \mathbb{R},$$

где

$$A = \begin{pmatrix} 1 & 1 & 9 & 5 \\ 3 & 5 & 0 & 8 \\ 2 & 0 & 6 & 13 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 7 \\ 3 \\ 5 \end{pmatrix}, \quad \vec{c} = \begin{pmatrix} 1 \\ 3 \\ 5 \\ 2 \end{pmatrix}.$$

Рис. 9: Задача

Векторизованные ограничения и целевая функция оптимизации

```
[12]: # Определение объекта модели с именем vector_model:
      vector_model = Model(GLPK.Optimizer)

[12]: A JuMP Model
      ├ solver: GLPK
      ├ objective_sense: FEASIBILITY_SENSE
      ├ num_variables: 0
      ├ num_constraints: 0
      └ Names registered in the model: none
```

Рис. 10: Определение объекта модели с именем vector_model

```
[13]: # Определение начальных данных:
      A= [ 1 1 9 5;
      3 5 0 8;
      2 0 6 13]
      b = [7; 3; 5]
      c = [1; 3; 5; 2]

[13]: 4-element Vector{Int64}:
       1
       3
       5
       2
```

Рис. 11: Определение начальных данных

```
[14]: # Определение вектора переменных:
      @variable(vector_model, x[1:4] >= 0)

[14]: 4-element Vector{VariableRef}:
       x[1]
       x[2]
       x[3]
       x[4]
```

Рис. 12: Определение вектора переменных

```
[15]: # Определение ограничений модели:
      @constraint(vector_model, A * x .== b)

[16]: 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
       x[1] + x[2] + 9 x[3] + 5 x[4] == 7
       3 x[1] + 5 x[2] + 8 x[4] == 3
       2 x[1] + 6 x[3] + 13 x[4] == 5
```

Рис. 13: Определение ограничений модели

Векторизованные ограничения и целевая функция оптимизации

```
[16]: # Определение целевой функции:
      @objective(vector_model, Min, c' * x)
```

[16]: $x_1 + 3x_2 + 5x_3 + 2x_4$

Рис. 14: Определение целевой функции

```
[17]: # Вызов функции оптимизации:
      optimize!(vector_model)
```

Рис. 15: Вызов функции оптимизации

```
[18]: # Определение причины завершения работы оптимизатора:
      termination_status(vector_model)
```

```
[18]: OPTIMAL::TerminationStatusCode = 1
```

Рис. 16: Определение причины завершения работы оптимизатора

```
[19]: # Демонстрация результата оптимизации:
      @show objective_value(vector_model);
```

```
objective_value(vector_model) = 4.9230769230769225
```

Рис. 17: Результат

Оптимизация рациона питания

Задача

Рассмотрим применение DenseAxisArrays на примере решения задачи оптимизации рациона питании в заведении быстрого питания при условии, что задано ограничение на количество потребляемых калорий (1800–2200), белков (>= 91), жиров (0–65) и соли (0–1779), а также перечень определённых продуктов питания с указанием их стоимости — гамбургер (2.49 ден.ед.), курица (2.89 ден.ед.), сосиска в тесте (1.50 ден.ед.), жареный картофель (1.89 ден.ед.), макароны (2.09 ден.ед.), пицца (1.99 ден.ед.), салат (2.49 ден.ед.), молочный коктейль (0.89 ден.ед.), мороженное (1.59 ден.ед.). Также известно содержание калорий, белков, жиров и соли в указанных продуктах.

Оптимизация рациона питания

```
[21]: # Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray(
    [1800 2200;
    91 Inf;
    0 65;
    0 1779],
    ["calories", "protein", "fat", "sodium"],
    ["min", "max"])
```

```
[21]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, ["calories", "protein", "fat", "sodium"]
    Dimension 2, ["min", "max"]
And data, a 4×2 Matrix{Float64}:
 1800.0  2200.0
   91.0     Inf
    0.0    65.0
    0.0  1779.0
```

Рис. 18: Создание контейнера JuMP для хранения инфо об ограничениях

```
[22]: # массив данных с наименобаниями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza","salad", "milk", "ice cream"]
```

```
[22]: 9-element Vector{String}:
 "hamburger"
 "chicken"
 "hot dog"
 "fries"
 "macaroni"
 "pizza"
 "salad"
 "milk"
 "ice cream"
```

Рис. 19: Массив данных с наименованиями

```
[23]: # Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray(
[2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
foods)
```

```
[23]: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.49
 2.89
 1.5
 1.89
 2.09
 1.99
 2.49
 0.89
 1.59
```

Рис. 20: Массив стоимости продуктов

```
[24]: # Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray(
[410 24 26 730;
420 32 10 1190;
560 20 32 1800;
380 4 19 270;
320 12 10 930;
320 15 12 820;
320 31 12 1230;
100 8 2.5 125;
330 8 10 180],
foods,
["calories", "protein", "fat", "sodium"])
```

```
[24]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
    Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9×4 Matrix{Float64}:
 410.0  24.0  26.0   730.0
 420.0  32.0  10.0  1190.0
 560.0  20.0  32.0  1800.0
 380.0   4.0  19.0   270.0
 320.0  12.0  10.0   930.0
 320.0  15.0  12.0   820.0
 320.0  31.0  12.0  1230.0
 100.0   8.0   2.5   125.0
 330.0   8.0  10.0   180.0
```

Оптимизация рациона питания

```
[25]: # Определение объекта модели с именем model:
      model = Model(GLPK.Optimizer)
```

```
[25]: A JuMP Model
      ├ solver: GLPK
      ├ objective_sense: FEASIBILITY_SENSE
      ├ num_variables: 0
      ├ num_constraints: 0
      └ Names registered in the model: none
```

Рис. 22: Определение объекта модели

```
[26]: # Определим массив:
      categories = ["calories", "protein", "fat", "sodium"]
```

```
[26]: 4-element Vector{String}:
       "calories"
       "protein"
       "fat"
       "sodium"
```

```
[27]: # Определение переменных:
      @variables(model, begin
      category_data[c, "min"] <= nutrition[c = categories] <= category_data[c, "max"]
      # Сколько покупать продуктов:
      buy[foods] >= 0
      end)
```

```
[27]: (1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
          Dimension 1, ["calories", "protein", "fat", "sodium"]
      And data, a 4-element Vector{VariableRef}:
       nutrition[calories]
       nutrition[protein]
       nutrition[fat]
       nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
          Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
      And data, a 9-element Vector{VariableRef}:
       buy[hamburger]
       buy[chicken]
       buy[hot dog]
       buy[fries]
       buy[macaroni]
       buy[pizza]
       buy[salad]
       buy[milk]
       buy[ice cream])
```

Рис. 24: Определение переменных

```
[28]: # Определение целевой функции:
      @objective(model, Min, sum(cost[f] * buy[f] for f in foods))
```

[28]: $2.49buy_{hamburger} + 2.89buy_{chicken} + 1.5buy_{hotdog} + 1.89buy_{fries} + 2.09buy_{macaroni} + 1.99buy_{pizza} + 2.49buy_{salad} + 0.89buy_{milk} + 1.59buy_{icecream}$

Рис. 25: Определение целевой функции

Оптимизация рациона питания

```
[32]: # Определение ограничений модели:
      @constraint(model, [i in categories],
      sum(food_data[f, i] * buy[f] for f in foods) >= nutrition[i]
```

```
[33]: # Одномерный DenseAxisArray[Constraint(AffExpr(0.0), MathOptInterface.GreaterThan(0.0)), Constraint(AffExpr(2.0*buy[hot dog] + 2.0*buy[pizza] + 2.0*buy[salad] + 2.0*buy[ice cream] + 2.0*buy[milk]), MathOptInterface.GreaterThan(2.0)), Constraint(AffExpr(0.0), MathOptInterface.GreaterThan(0.0)), Constraint(AffExpr(0.0), MathOptInterface.GreaterThan(0.0))] with 4 elements:
      [calories] : 0.0 ≤ 0.0
      [cholesterol] : 2.0 ≤ 2.0*buy[hot dog] + 2.0*buy[pizza] + 2.0*buy[salad] + 2.0*buy[ice cream] + 2.0*buy[milk]
      [protein] : 0.0 ≤ 0.0
      [vitamins] : 0.0 ≤ 0.0
```

Выводы

Вывод

В результате выполнения лабораторной работы, я освоил пакеты Julia для решения задач оптимизации.